Dynamic Partial Reconfiguration

Arun Murugappan, Pranav T Varkey, Shrikrishna Pandit

Table of contents

- Fast and Flexible FPGA Development using **Hierarchical Partial Reconfiguration**(2022)
- **RapidStream 2.0**: Automated Parallel Implementation of Latency–Insensitive FPGA Designs Through Partial Reconfiguration(2022, 2023)
- **ZyPR**: End-to-end Build Tool and Runtime Manager for Partial Reconfiguration of FPGA SoCs at the Edge (2022)

Toolflows

Hierarchical Partial Reconfiguration

Fast and Flexible FPGA Development.

- Fast and Flexible FPGA Development using **Hierarchical Partial Reconfiguration**(2022)
  - Authors: Dongjoon Park, Yuanlong Xiao and Andr´e DeHon. University of Pennsylvania
  - Conference: 2022 International Conference on Field-Programmable Technology (ICFPT) IEEE

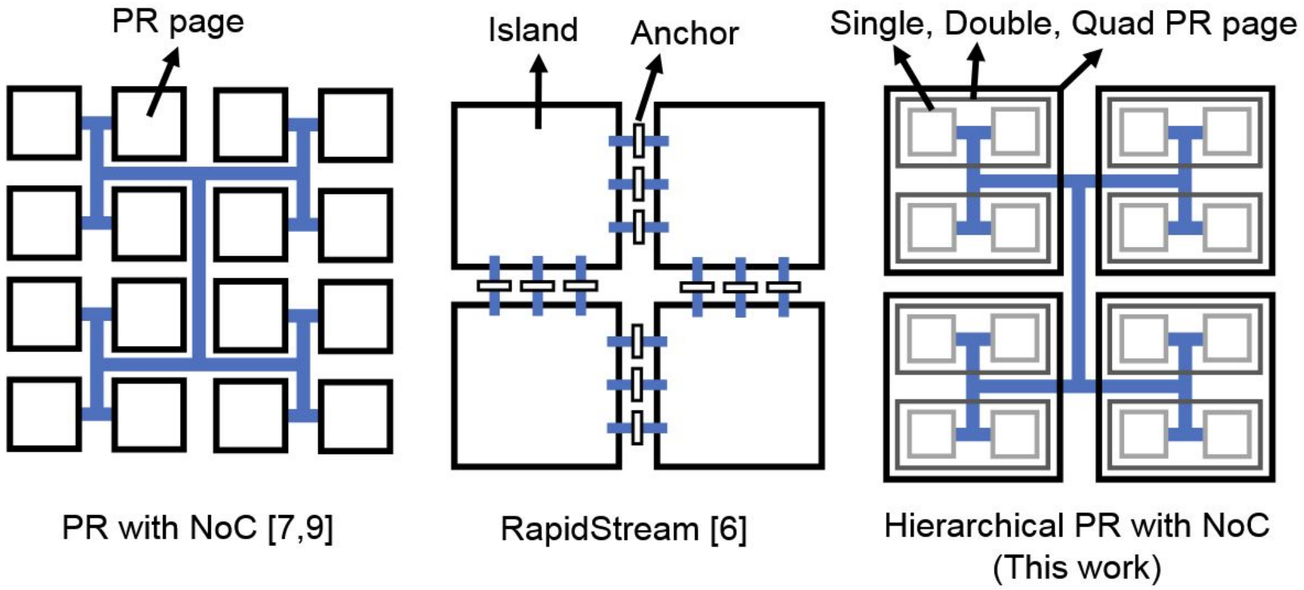
PLD: Partition Linking LoaDing on Programmable Logic Devices

- PLD: Fast FPGA Compilation to Make Reconfigurable Acceleration Compatible with Modern Incremental Refinement Software Development.
  - Authors: Yuanlong Xiao, Eric Micallef, Andrew Butt, Matthew Hofman, Marc Alston, Matthew Goldsmith, Andrew Merczynski-Hait, Andre DeHon
  - Conference: ASPLOS 2022

Hierarchical Partial Reconfiguration

Hierarchical Partial Reconfiguration:

- FPGA compilation is notoriously slow.
- Existing solutions force users to break designs into small, fixed regions, often resulting in suboptimal performance or extra manual effort.
- Unlike prior approaches with rigid, fixed-sized regions, the method this paper follows, allows, flexible, hierarchical regions that adapt to the size of each design block.



Hierarchical Partial Reconfiguration

C/C++ Kernel Source (HLS)

↓

Vitis HLS Compilation
(Generates RTL

↓

Resource Estimation
PLD Page Pages (PP)
- Hierarchical PR (HPR)*
  Greedy Algorithm (HPR)

↓

Vivado Implementation
(Synthesis, Place & Route)

↓

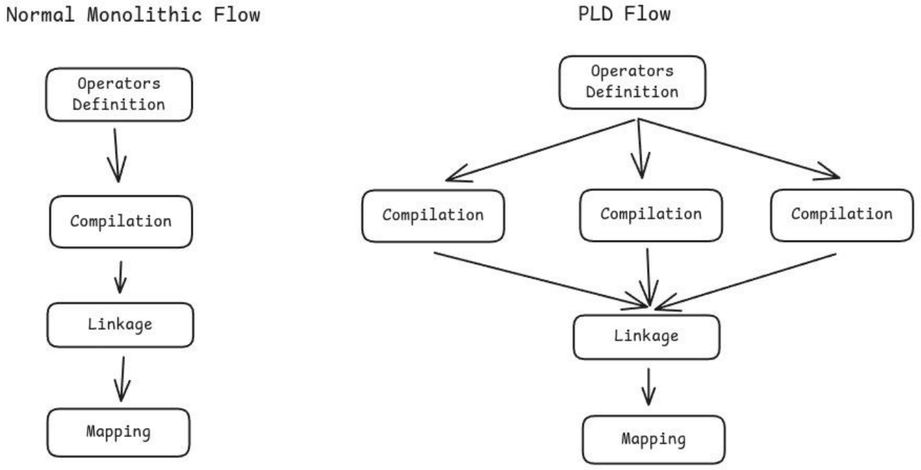
Partial Bitstream Generation
- NoC generation & page interconnects
- Abstract Shell for each PR page (HPR)

↓

Host Code & Software Linking
(DMA engine, config drivers)
Uses Vitis APIs

↓

FPGA Bitstream
(Overlay + PR Region)



Hierarchical Partial Reconfiguration

PLD: Fast FPGA Compilation to Make Reconfigurable Acceleration Compatible with Modern Incremental Refinement Software Development.

- This paper develops the framework on which the HPR paper builds.
- It introduces 3 ways to compile a HLS code.

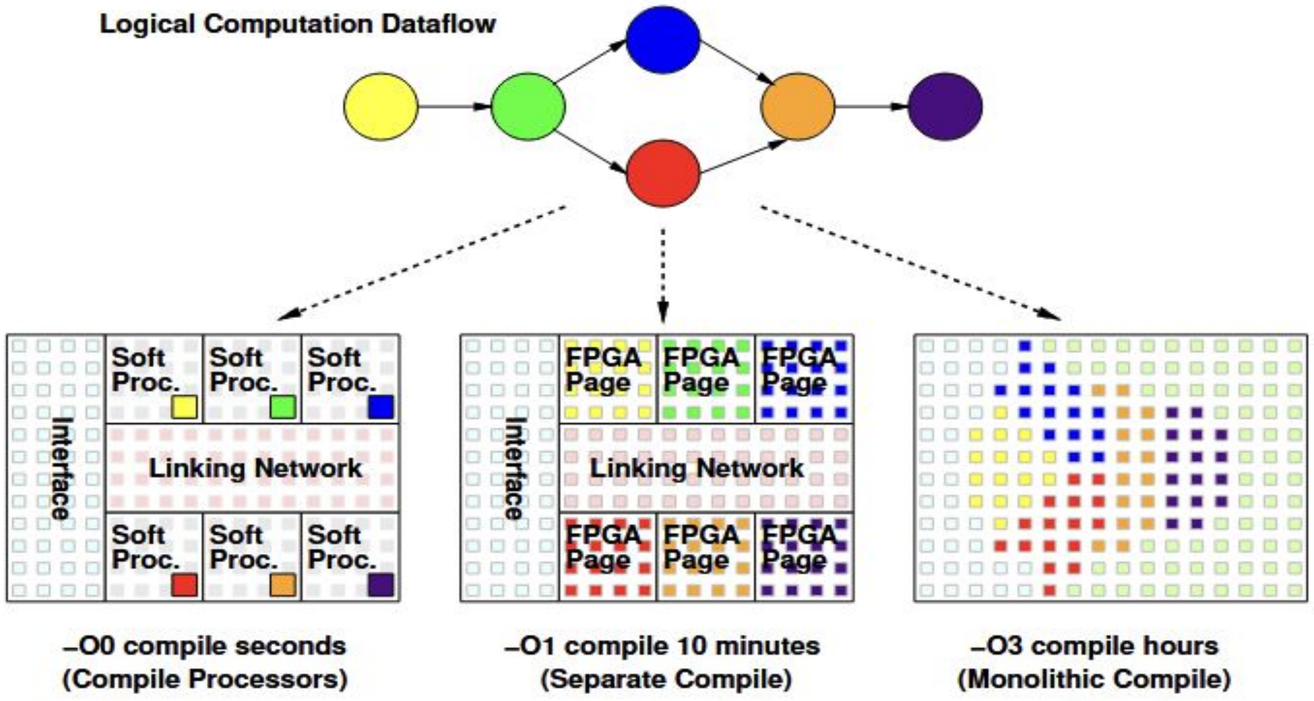


Figure 1: Fast and Separate Compilation Mapping Strategies

PLD

Pages: In PLD, a single operator PR region is called a page which made as an analogy with a virtual memory pages.

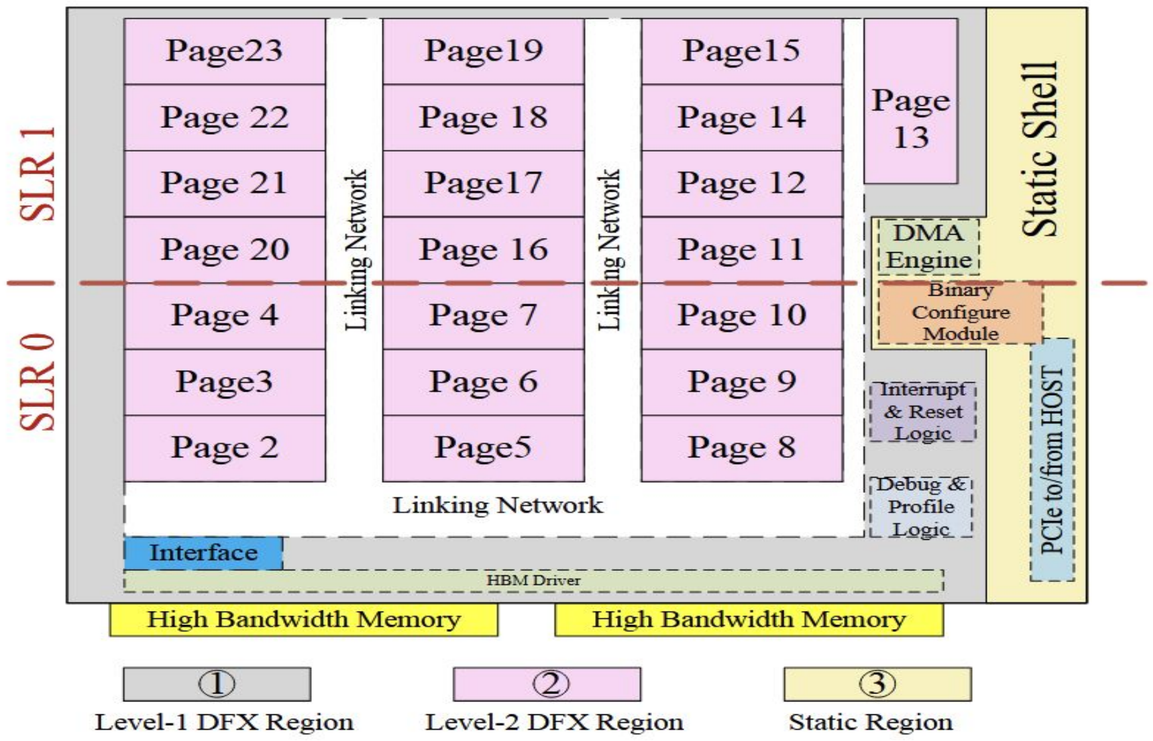


Figure 3: PLD FPGA Decomposition: Pages, Linking Network, and Support Infrastructure

PLD

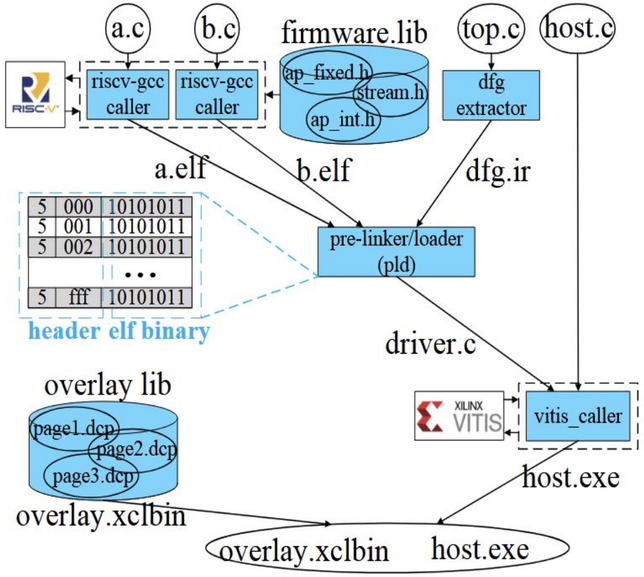


Figure 5: -O0 Compile Flow

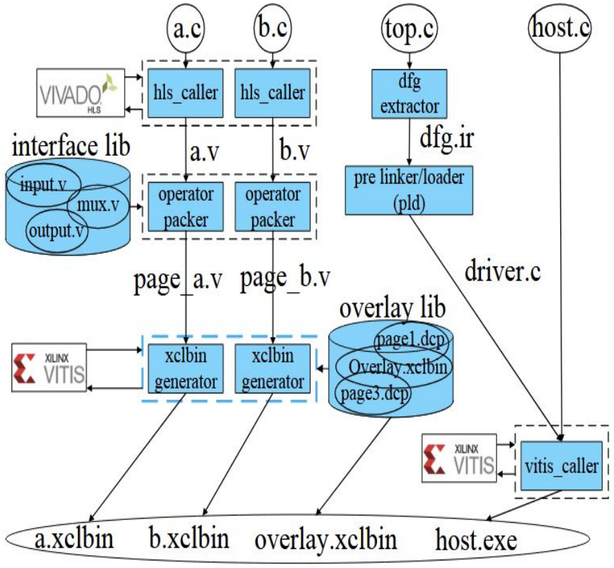


Figure 6: -O1 Compile Flow

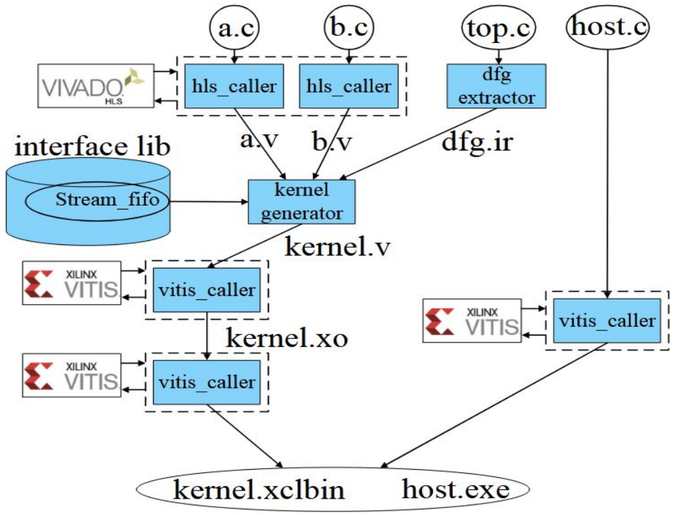


Figure 7: -O3 Compile Flow

PLD

Rules of Decomposition:

- **Operator Discipline:**
  - **Streaming** over Global Shared Memory.
  - Avoid Dynamic Allocation or Recursion, Compatible Datatypes
  - Conditional Processor-Only Operations.
- **Operator Decomposition:**
  - Modular Computational Blocks.
  - Stream-Connected Communication,
  - Logical Decoupling
  - Optional for better performance
    - Merge Operators which otherwise suffer from I/O bottlenecks in hls
    - Split Operators to exploit data parallelism

Toolflows - Fast and flexible

Hierarchical Partial Reconfiguration (HPR) Framework:

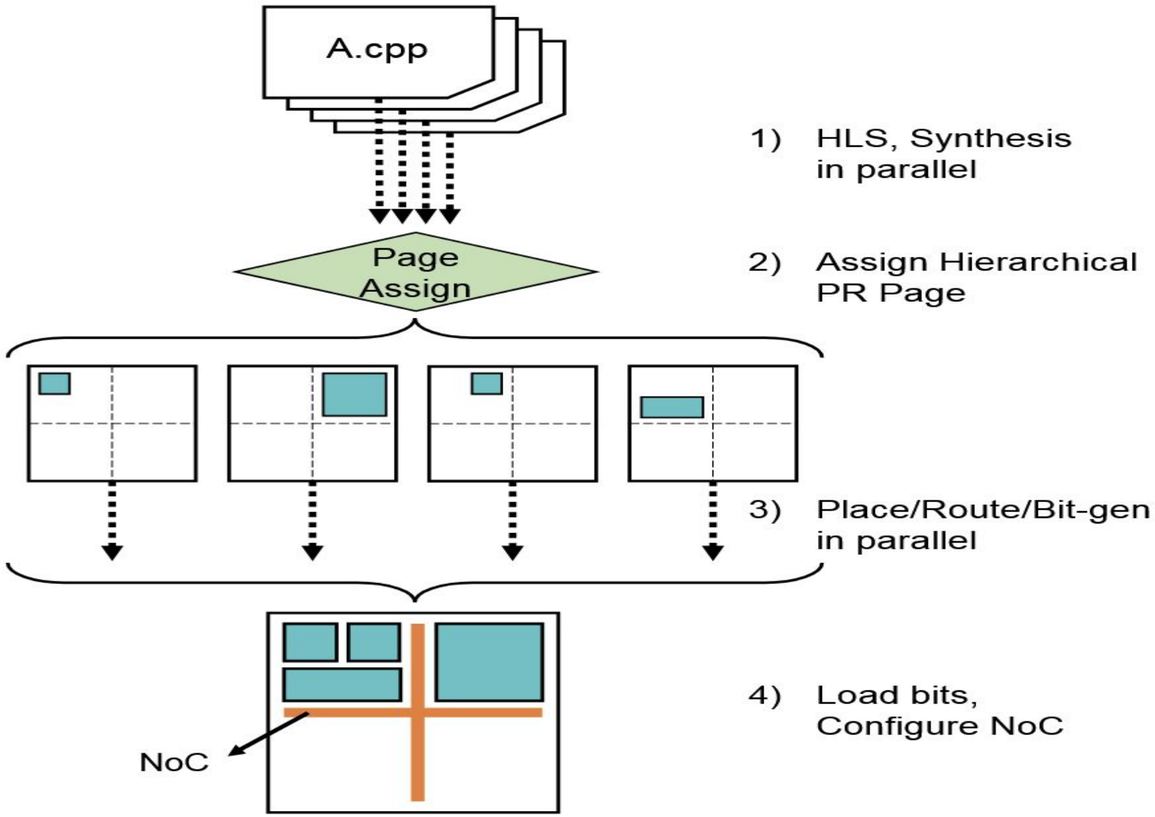
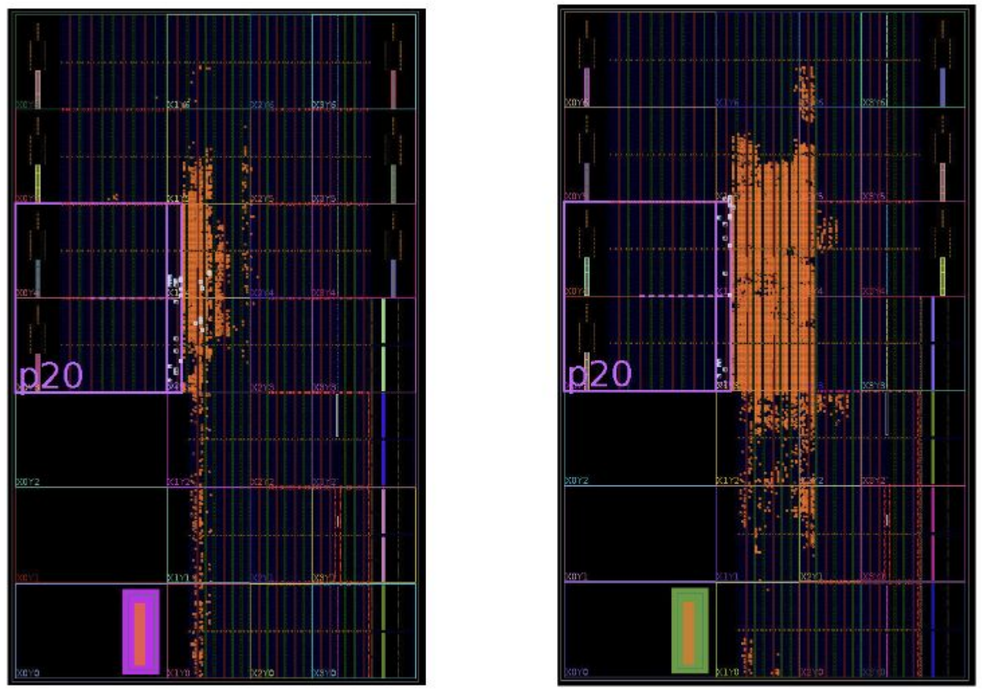


Fig. 2. Separate Compilation with Hierarchical PR Pages Overview

Fast and flexible



(a) p20 with p_NoC

(b) p20 without p_NoC

Fig. 4. Abstract shells for p20 comparison. Orange elements are the static resources showing the extent of the expanded regions.

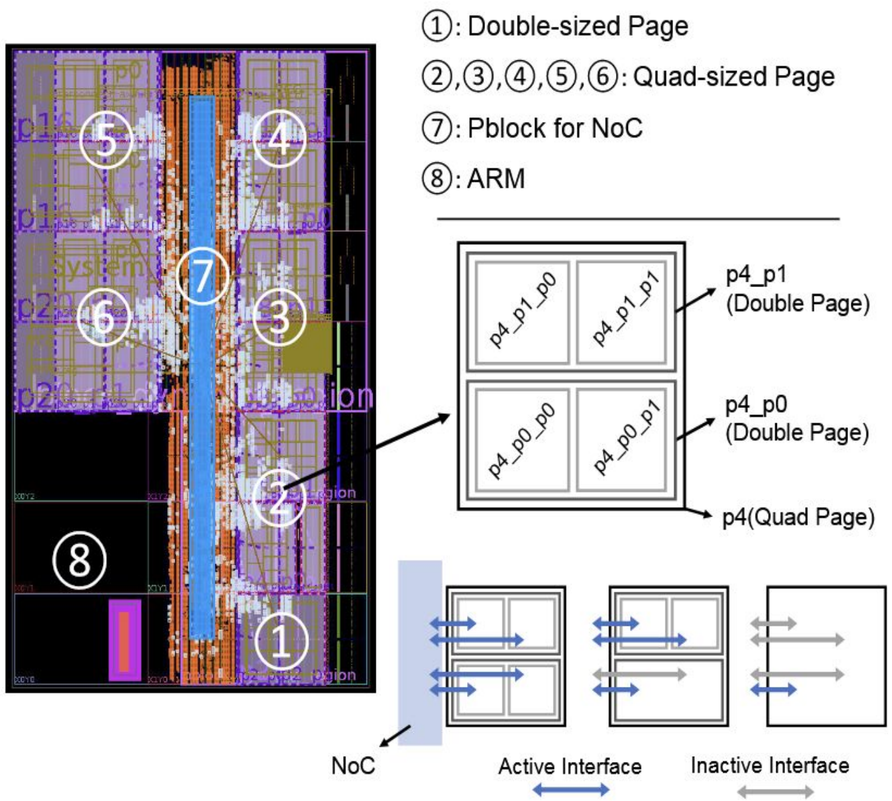
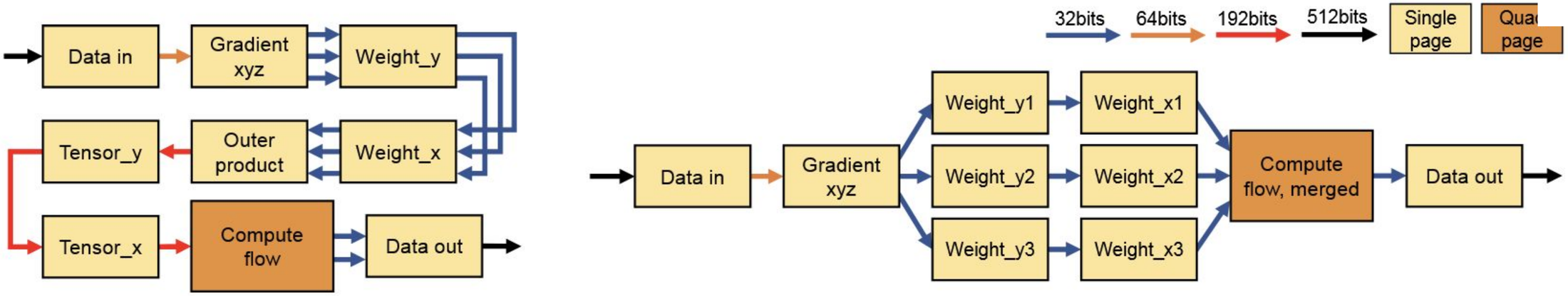


Fig. 3. Example Overlay on the Xilinx ZU9EG FPGA

Fast and flexible



(a) Natural Decomposition of Optical Flow

(b) Decomposition of Optical Flow after Incremental Development

Fig. 5. Optical Flow Dataflow Graph

TABLE III
OPTICAL FLOW INCREMENTAL DEVELOPMENT WITH HIERARCHICAL PR PAGES

Steps	Largest Operator	Page Size	Compile Jobs	Largest Operator Resource Usage			Incremental Compile Time	App Latency
				LUTs	B18s†	DSPs		
1) Natural	Compute flow	Quad	9	16829	7	24	**261s**	18.7ms
2) Merge Tensor_x	Compute flow	Quad	1	17560	7	54	**245s**	18.1ms
3) Merge Tensor_y	Compute flow	Quad	1	18473	33	68	**274s**	16.0ms
4) Merge Outer prod.	Compute flow	Quad	1	20357	39	74	**288s**	14.7ms
5) Split Weight_x	Weight_x1	Single	3	1690	6	10	**94s**	14.7ms
6) Split Weight_y	Weight_y1	Single	3	1791	18	10	**107s**	14.7ms
7) Optimize Compute flow	Compute flow	Quad	1	20307	45	78	**291s**	8.7ms
8) Change data type	Compute flow	Quad	1	12471	49	192	**274s**	8.7ms

† B18s: BRAM18s

"App Latency" – application execution time per input

Fast and flexible

TABLE V
ROSETTA BENCHMARK WITH PR PAGES

Benchmarks	Resource Usage			Usage by Page Size			Compile Time	Compile Speedup over Mono.	App Latency	App Latency Improvement over Mono.	App Latency Improvement over Fixed-Sized Page
	LUTs	B18s	DSPs	Sngl.	Dbl.	Qd.					
Optical, (64,mix)*	35953	218	168	17	0	0	276s	N/A	32.1ms	0.6	1.0*
Optical, (96,mix)*	40765	222	330	17	0	0	329s	2.2	43.4ms	0.4	1.0*
Optical, (64,fixed pt)	36336	160	148	9	0	**1**	322s	**2.2**	8.8ms	2.2	**3.6**
Optical, (96,mix)	28500	164	262	9	0	**1**	305s	**2.3**	8.8ms	2.2	**4.9**
Rendering, Par=1*	8605	94	9	5	0	0	151s	2.8	2.6ms	1.0	1.0*
Rendering, Par=2	22435	106	18	6	**1**	0	169s	**2.5**	1.8ms	1.4	**1.4**
Digit Rec, Par=40*	44774	381	2	10	0	0	212s	4.3	7.0ms	1.7	1.0*
Digit Rec, Par=80	70638	701	2	0	**10**	0	251s	**5.3**	4.7ms	2.4	**1.5**
Spam, Par=32*	51461	204	256	15	0	0	287s	2.6	72.5ms	0.5	1.0*
Spam, Par=64	57263	198	512	7	**7**	0	307s	2.8	72.4ms	0.4	1.0

* Decompositions that can also be mapped to the Fixed-Sized PR Pages.

- The Rosetta HLS benchmark was used to get the above results. The platform used for testing was the ZCU102 embedded platform according to the paper.

Toolflows - Fast and flexible

Aspect	**Prior Work (Fixed PR)**	**This Work (Hierarchical PR)**
Region Size	Fixed	Variable (Single/Double/Quad)
Manual Effort	High (manual split)	Low (automatic assignment)
Performance	Moderate	1.4–4.9× improvement
Compile Time	Hours	2–5 minutes (2.2–5.3× faster)
Flexibility	Low	High

PHD Thesis: SOFTWARE-LIKE INCREMENTAL REFINEMENT ON FPGA USING PARTIAL RECONFIGURATION

ISFPGA' 24: REFINE– Runtime Execution Feedback for INcremental Evolution on FPGA Designs

RapidStream

RapidStream 1.0

Authors : Licheng Guo, Pongstorn Maidee, Yun Zhou, Chris Lavin, Jie Wang, Yuze Chi, Weikang Qiao, Alireza Kaviani, Zhiru Zhang, Jason Cong

Conference :FPGA '22: The 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays

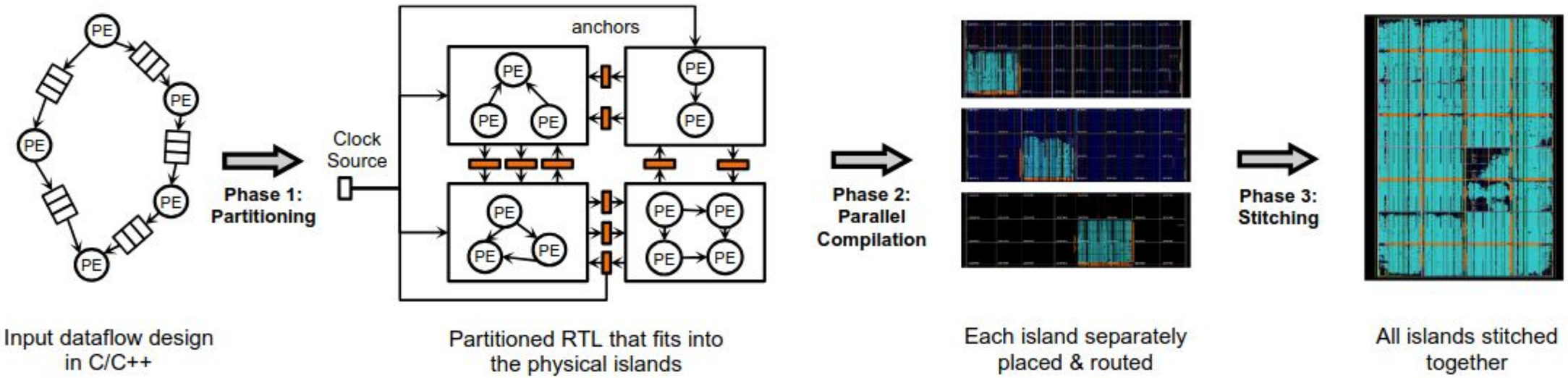
RapidStream 2.0

Authors: YUZE CHI, and LINGHAO SONG, YUANLONG XIAO, ALIREZA KAVIANI, ZHIRU ZHANG, JASON CONG

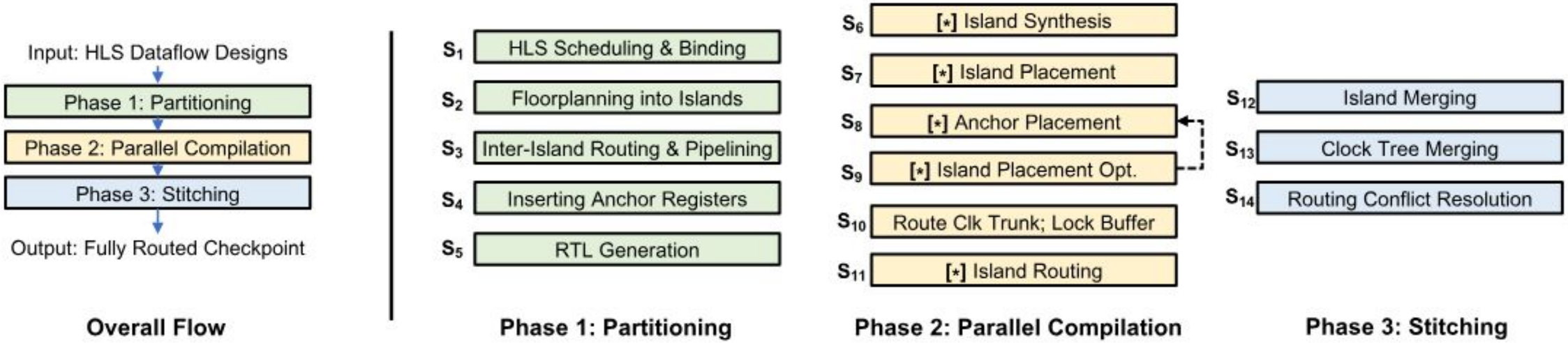
Publication: ACM TRETS (2023)

RapidStream 1.0

- RapidStream, a split compilation flow featuring tight integration of HLS-level pipelining and physical design to accelerate the end-to-end FPGA compilation
- It achieves this by:
  - Partitioning the design into Islands and anchors, enabling parallel placement and routing



RapidStream 1.0



RapidStream 1.0

- Phase 1: Partitioning
  - Mapping PEs to Islands
  - Global routing & Pipelining Inter-Island Connections (fig A)
  - Inserting anchor registers (fig B)
- Phase 1 produces an optimized RTL that is floorplanned to island regions and anchor regions

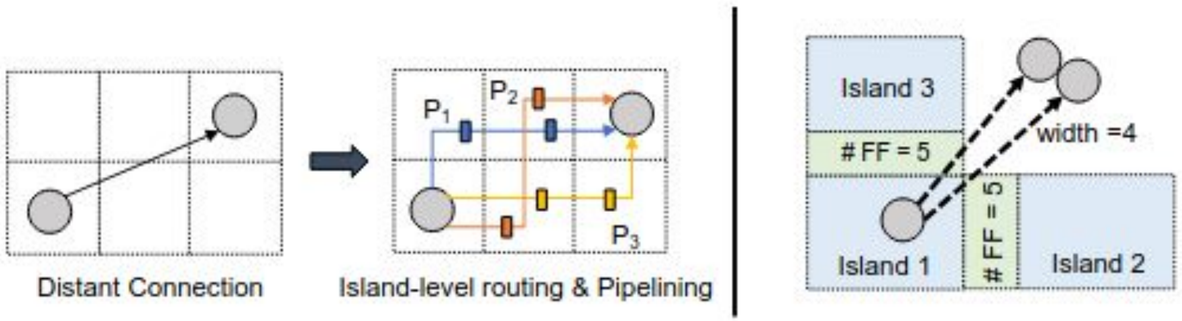


Fig A

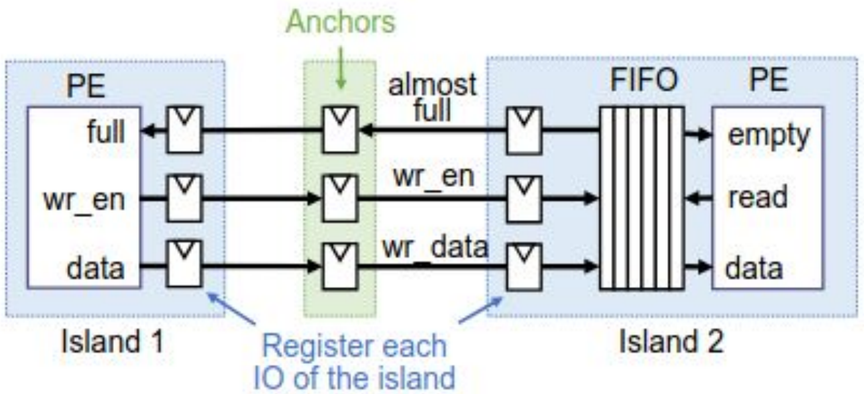


Fig B

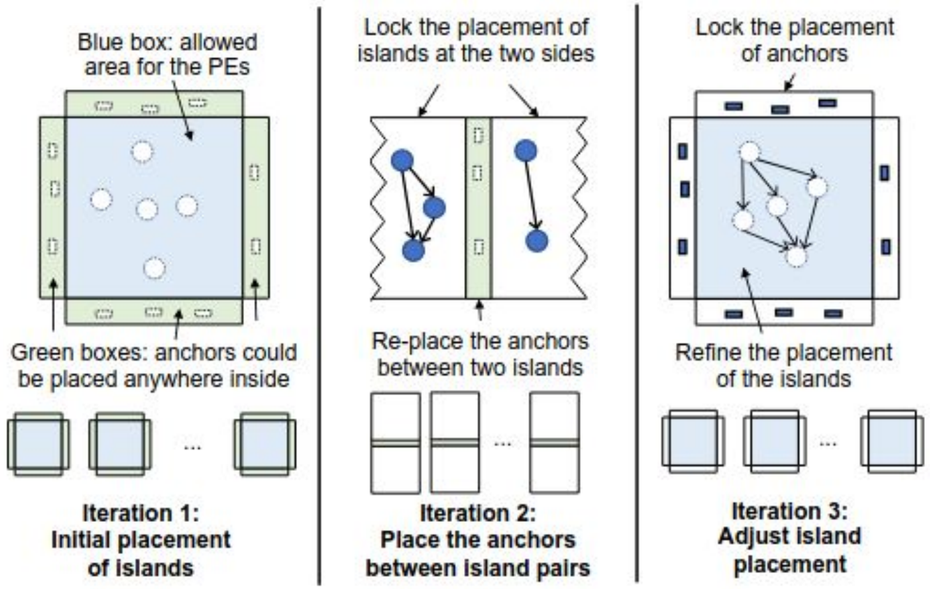
RapidStream 1.0

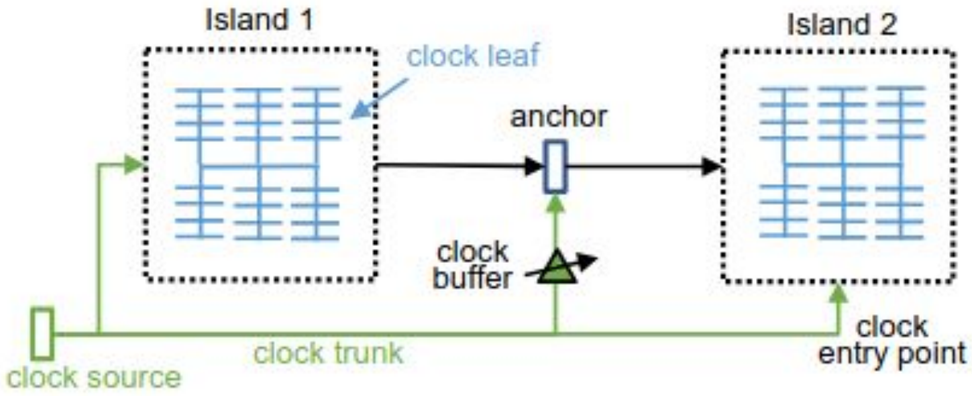
- ILP formulation for PE mapping : the goal is to map each v ∈ V to one unique island such that the resource of each island is not overused and the total wirelength is minimized. We use the weighted Manhattan distance to calculate the total wirelength

$$\sum_{e_{ij} \in E} e_{ij}.width \times (|v_i.row - v_j.row| + |v_i.col - v_j.col|),$$

RapidStream 1.0

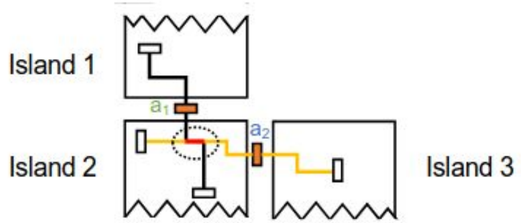
- Phase 2: Parallel Compilation
  - Iterative Placement of Anchors and Islands
    - Iteration 1 : Initial placement of islands
    - Iteration 2 : Each placer handles a pair of islands and places the anchor
    - Iteration 3 : Fine tune placement of islands based on anchor location
  - Clock Routing

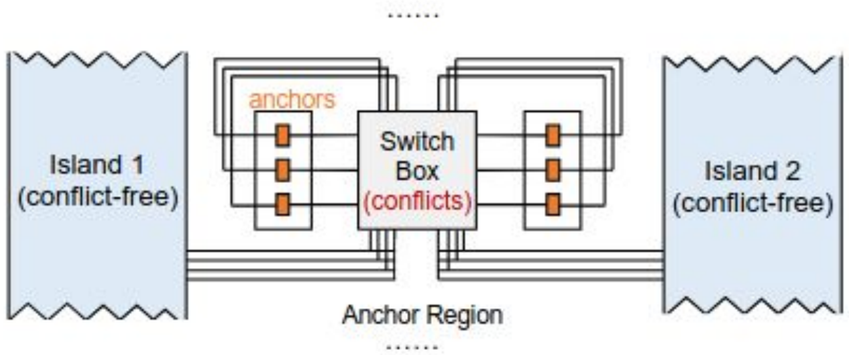


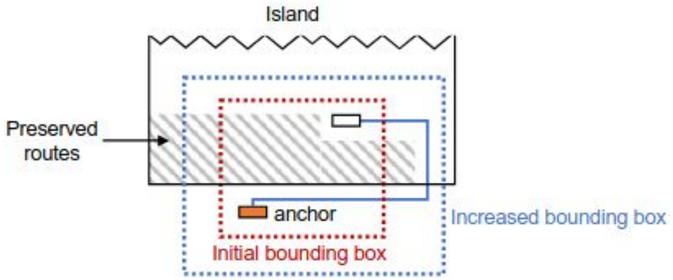


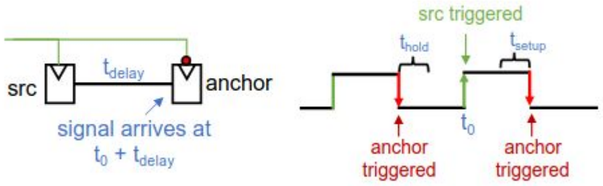
RapidStream 1.0

- Phase 3: STITCHING AND INTER-ISLAND ROUTING
  - Island Merging : RapidWright Framework to assemble physical information of islands
  - Inter-Island Routing
    - Solution 1 : Vivado Router
    - Solution 2 : Custom Partial route(RWroute)



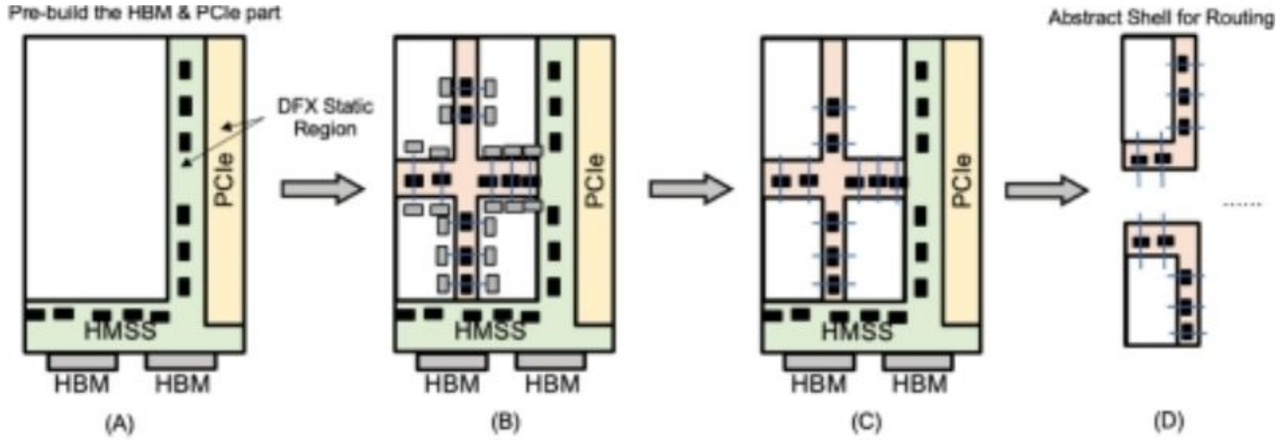




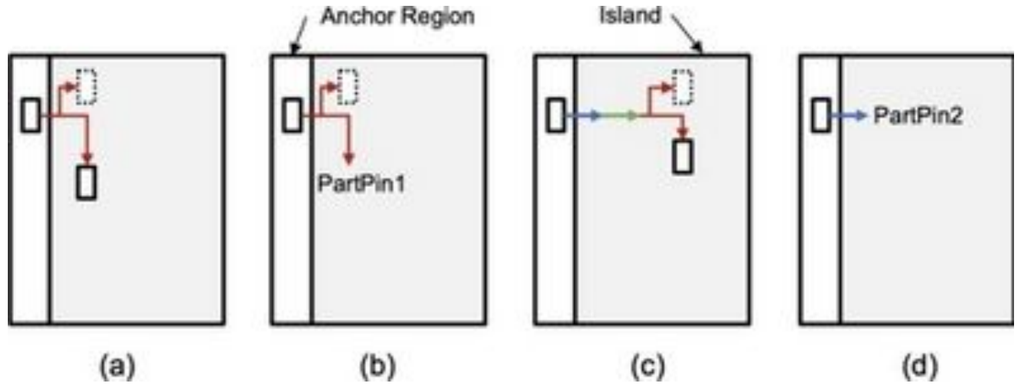


RapidStream 2.0

RapidStream 2.0 uses Vivado's DFX flow to provide more robust implementation



RapidStream 2.0 flow



Virtual Pins

RapidStream 1.0

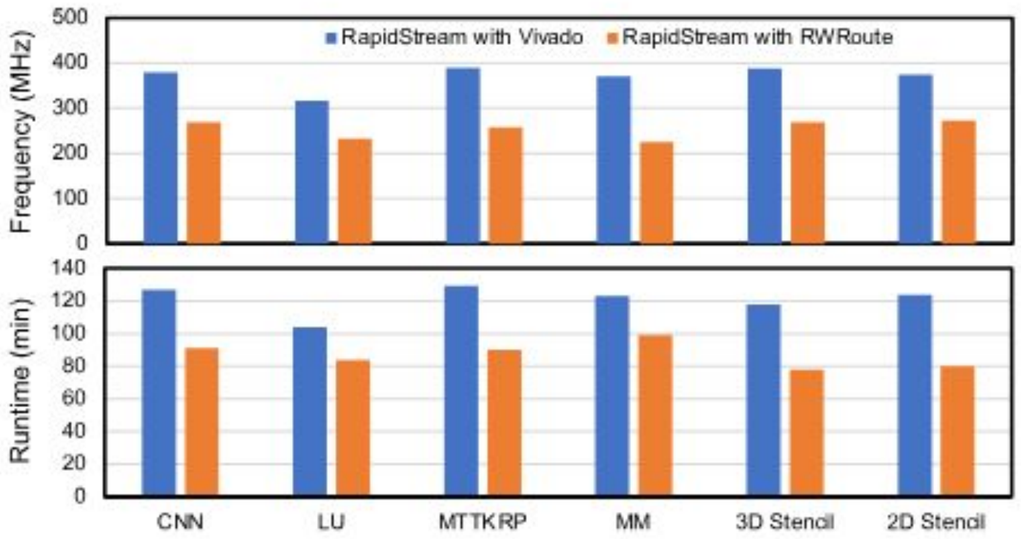
- Benchmarks:
  - RapidStream using four servers, each with the 56-core Intel Xeon E5-2680 v4 CPU at 2.40 GHz and 128 GB of memory.
  - Sources for benchmarks are from AutoSA project and SODA project
  - Implemented on U250

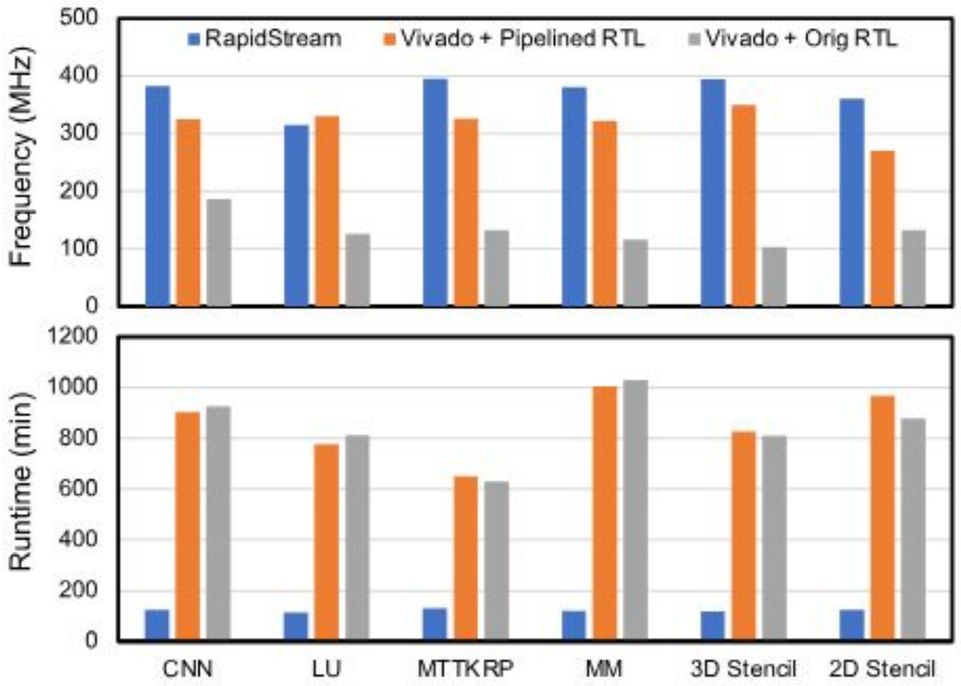
Name	# V	# E	Topology	DSP %	BRAM %	FF %	LUT %
MM	463	854	2-D Mesh	62	23	34	69
CNN	439	813	2-D Mesh	59	33	32	50
LU	1691	4483	Triangular	20	41	26	66
MTTKRP	360	760	2-D Mesh	66	33	30	48
2-D Stencil	266	1562	Irregular DAG	52	21	27	45
3-D Stencil	1314	2866	Irregular DAG	64	39	35	53

#V number of PEs
#E number of FF between PEs

RapidStream 1.0

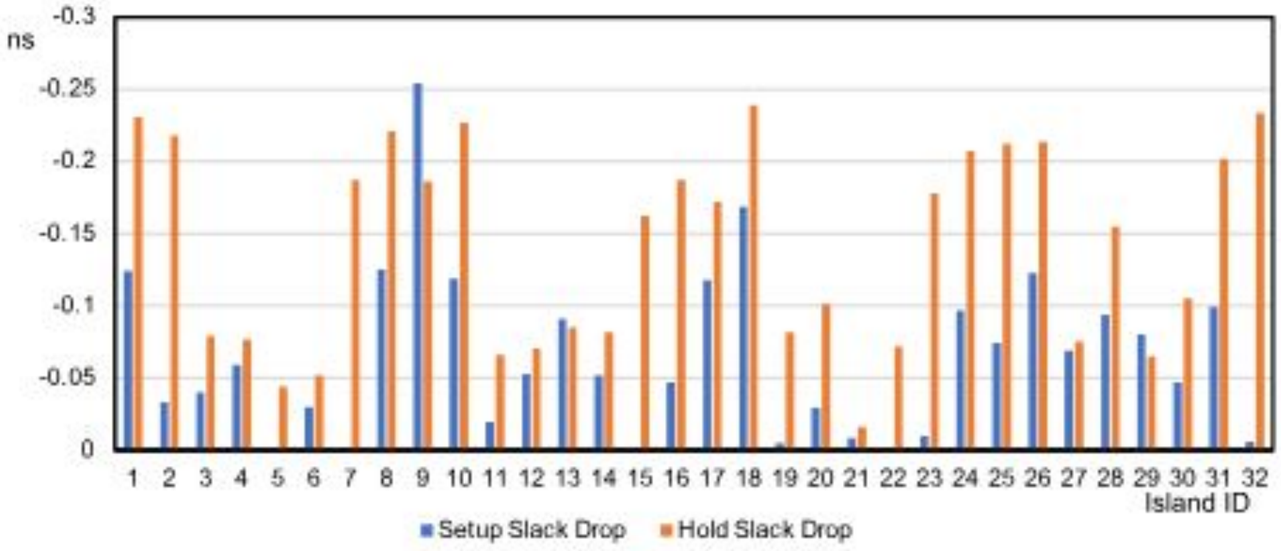
- Benchmarks:

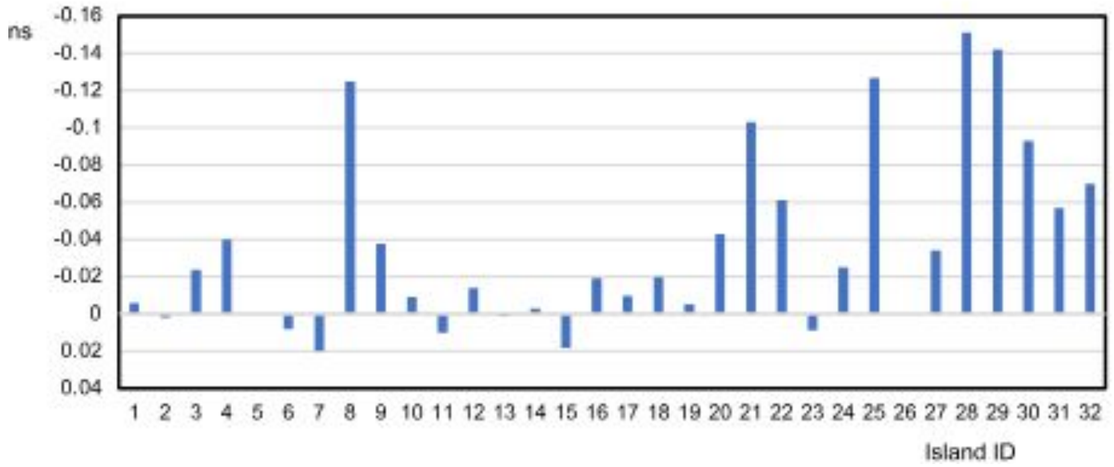




RapidStream 1.0

- Benchmarks:
  Timing degradation with and without constraints after stitching





RapidStream 2.0

- Benchmarks:

	LUT	FF	DSP	BRAM	# Task	# FIFO
gaussian-int	32%	14%	27%	46%	840	2440
gaussian-float	52%	40%	77%	13%	232	648

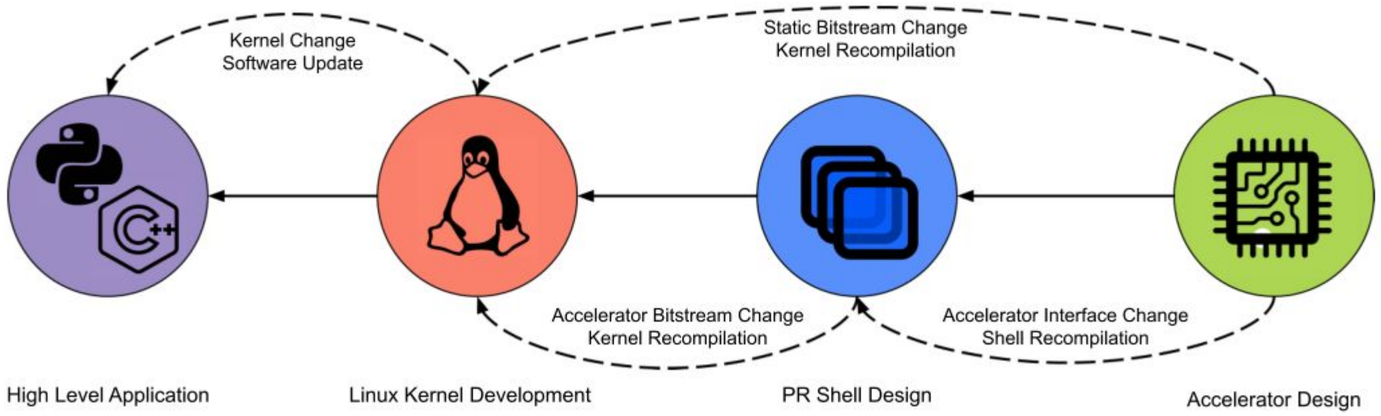
Test\Time (min)	Synthesis	Placement	Routing	Total Time	MHz
gaussian-int-vivado	74 (6.2×)	131 (6.6×)	243 (4.6×)	455 (5.1×)	268
gaussian-int-vivado-autobridge	80 (6.7×)	93 (4.7×)	143 (2.7×)	386 (4.3×)	300
gaussian-int-rapidstream	12	20	53	89	300
gaussian-float-vivado	196 (9.6×)	236 (6.7×)	454 (7×)	897 (7.5×)	237
gaussian-float-vivado-autobridge	208 (10.4×)	220 (6.3×)	248 (3.8×)	683 (5.7×)	300
gaussian-float-rapidstream	20	35	65	120	300

ZyPR

- An end-to-end **build tool and runtime manager** specifically for Partial Reconfiguration (PR) on FPGA SoCs at the Edge
  - Authors: Alex R. Bucknall, Suhaib A. Fahmy
  - Conference: ACM Transactions on Reconfigurable Technology and Systems, June 2023
- A specialised **abstraction tool for Zynq platforms** to enable **high performance PR applications on edge**
- It's a build tool which deploys HLS accelerators and provides a shell to interact with these accelerators from the linux userspace through provided APIs
- Increased throughput for PR bitstreams on the Zynq Ultrascale+ by **2×** and **5.4×** compared to **Xilinx's FPGA Manager**

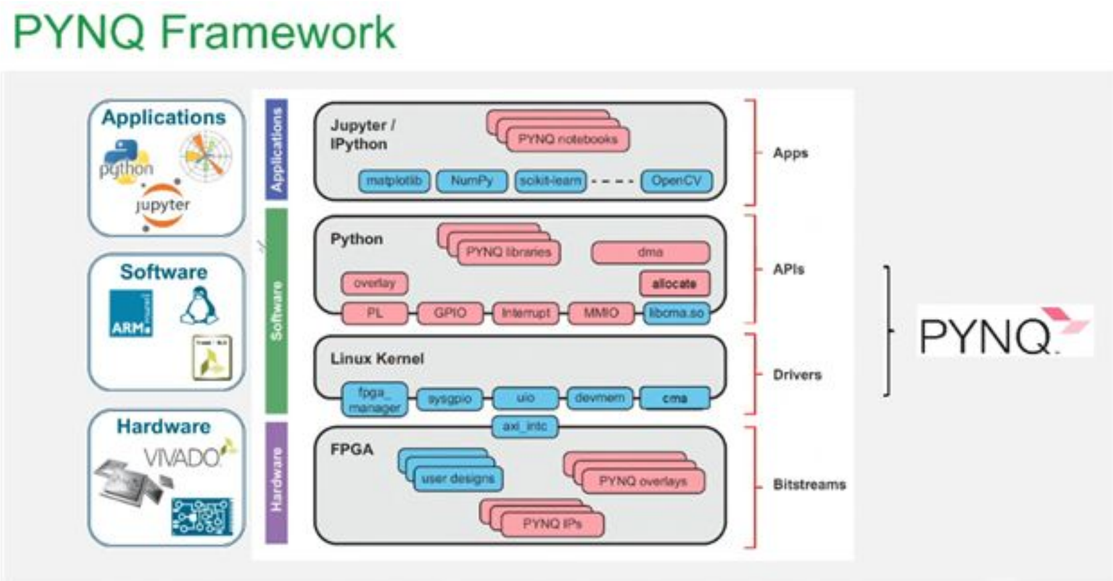
Tool Flow requirements

- Designing and building PR systems should be possible by **non-experts**
- Abstractions between hardware and software must be managed such that both PR and **accelerator performance are not impacted**
- Interacting with hardware accelerators should not require driver-level access—applications should run from **OS userspace**
- Fragmentation of vendor hardware should be managed by tooling; the tools should be **modular to support new architectures**.



Drawbacks of existing solutions

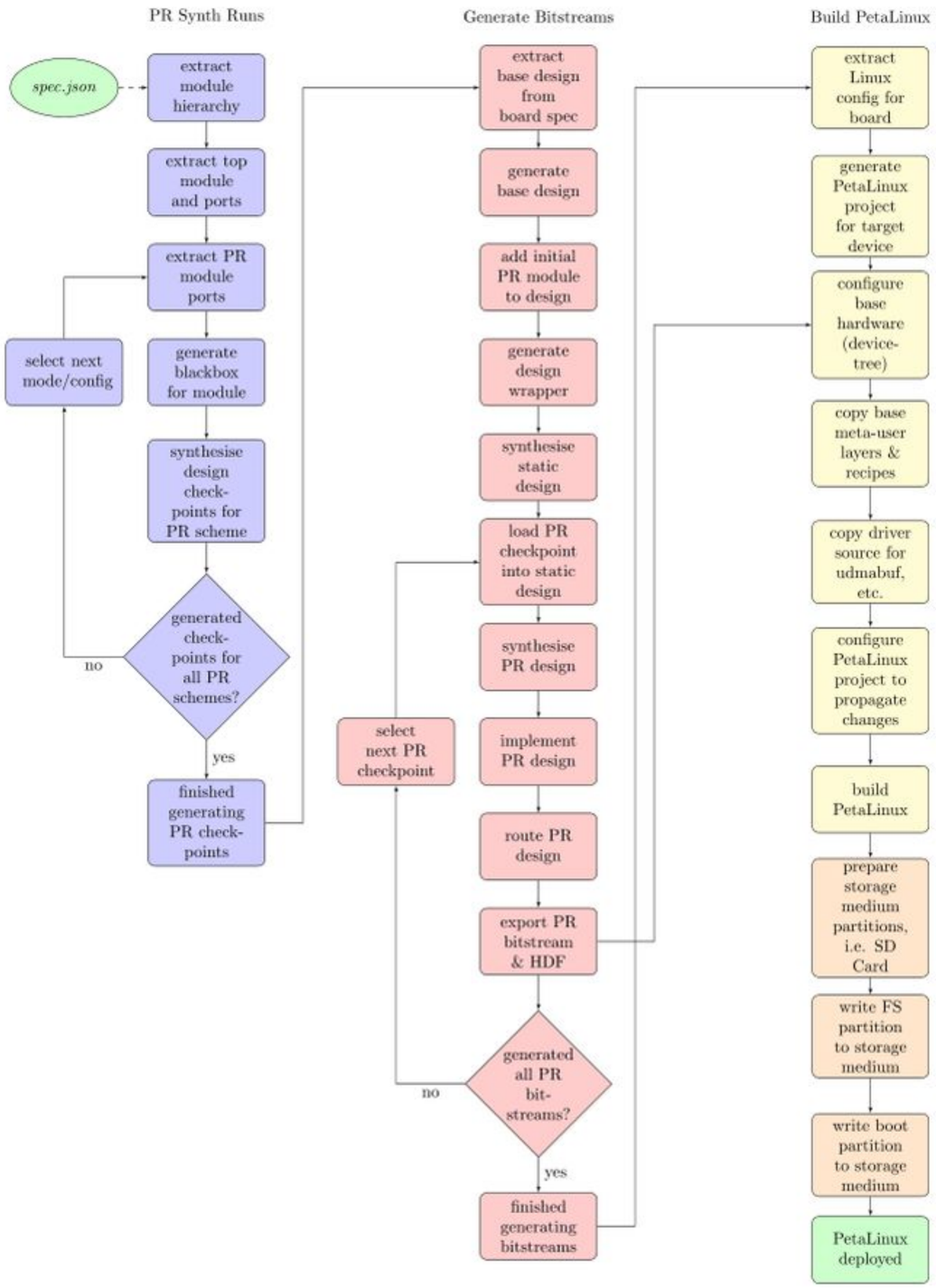
- **Pynq**:
  - Provides a shell for reconfiguration during runtime through python API calls
  - High abstraction: Integrates with existing python libraries and high abstraction level
  - Low performance: High reconfiguration time
- **FOS**:
  - Modularised the design flow of PR applications for the Zynq and ZynqMP
  - Need high expertise of each domain
- **FPGA Manager**:
  - Unsuitable for high-performance PR applications such as image processing or inline data streaming as the reconfiguration latency and throughput are comparatively low



Some Keywords

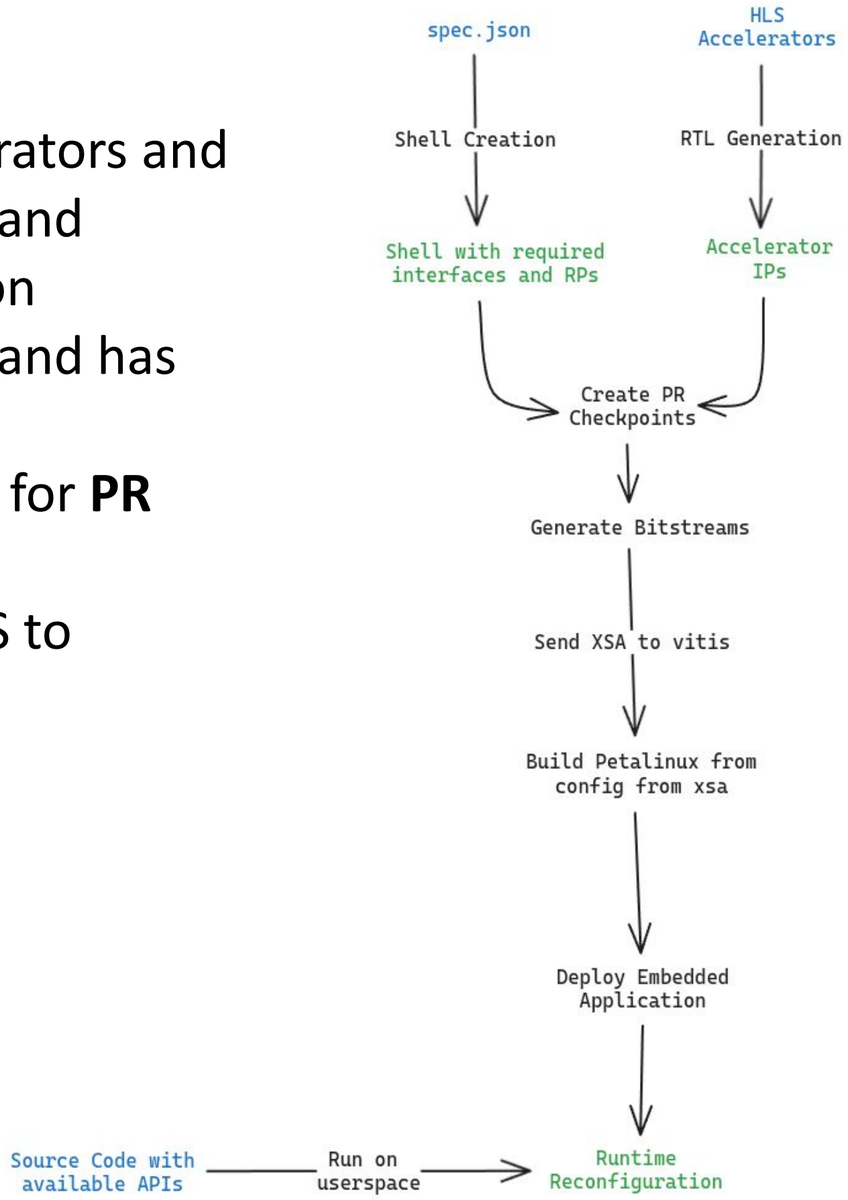
- **Device Trees**:
  - A tree that tracks all the hardware available to a kernel
  - Any change in the hardware would have required recompilation of the kernel
- **DTOs**:
  - Device tree overlays give users the ability to add and remove hardware nodes during runtime without recompilation
- **Petalinux**:
  - A minimalistic linux image designed for embedded system applications
- **MMIOs**:
  - Technique in which we can interact with hardware peripherals similar to reads and writes from memory
- **XSA**:
  - Compressed export directory that contains a map of IP cores that have accompanying Linux drivers
  - Used to create the device tree for the petalinux build
- **Userspace:**
  - Isolated (from kernel space) portion of system memory and execution environment where application software run
- **UIO**:
  - Userspace IO, a kernel layer that allows for direct communication to memory mapped devices from userspace

PR Build Flow Stages



What exactly is ZyPR?

- A **python cli** to manage generation of accelerators and their associated shell (supported interfaces) and drivers (petalinux) for runtime reconfiguration
- The designer can use their own pre-built IPs and has the ability to specify the pblock
- **Slot based architecture** of PRRs and support for **PR module chaining**
- The tool also creates custom DTOs for the OS to recognise hardware changes



ZyPR Components

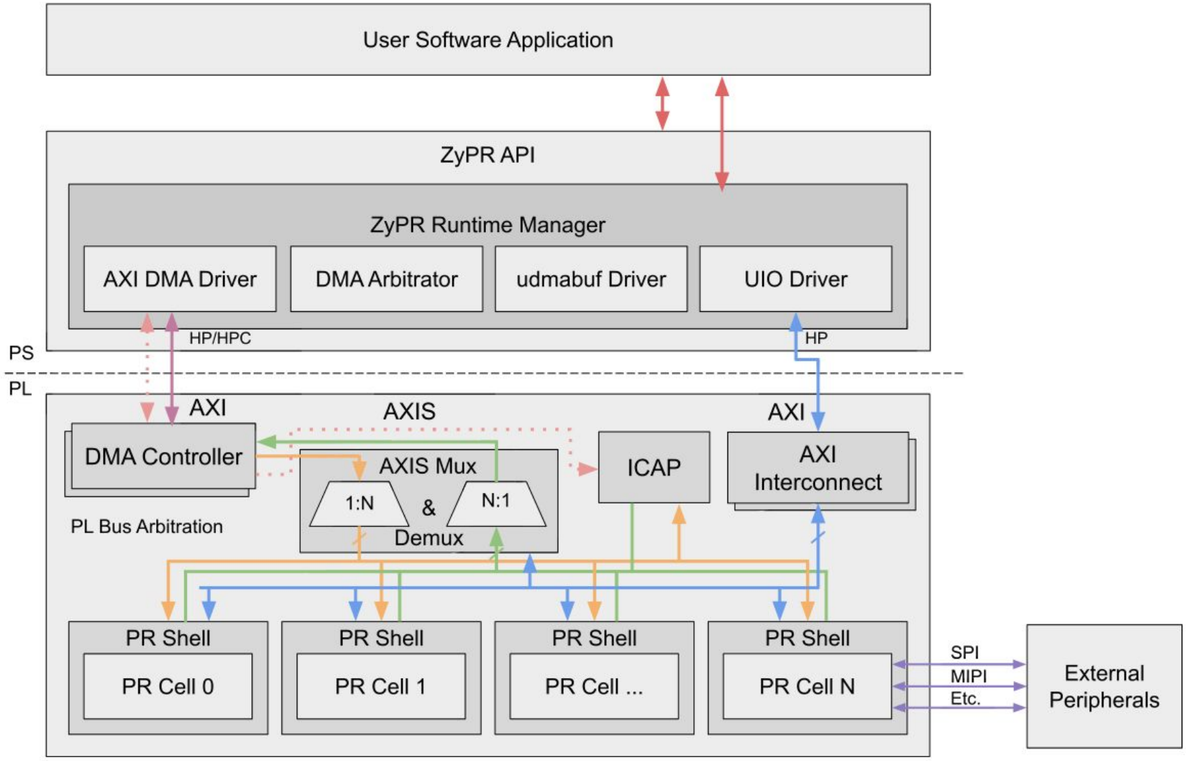


Fig. 5. PL architecture generated using the ZyPR build tooling.

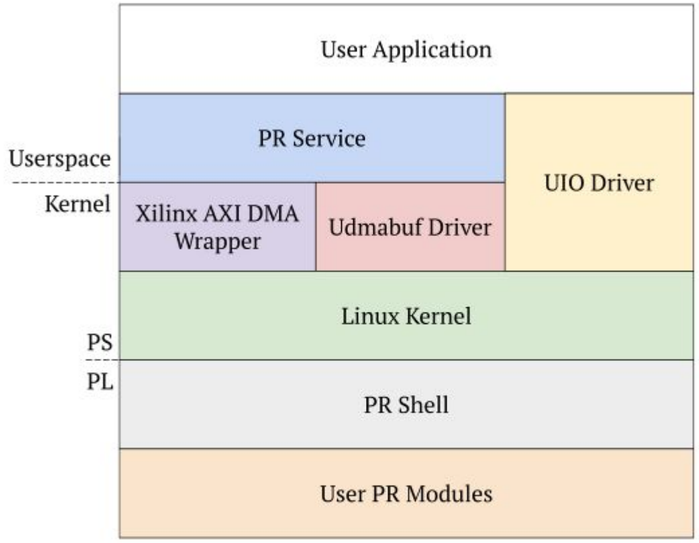
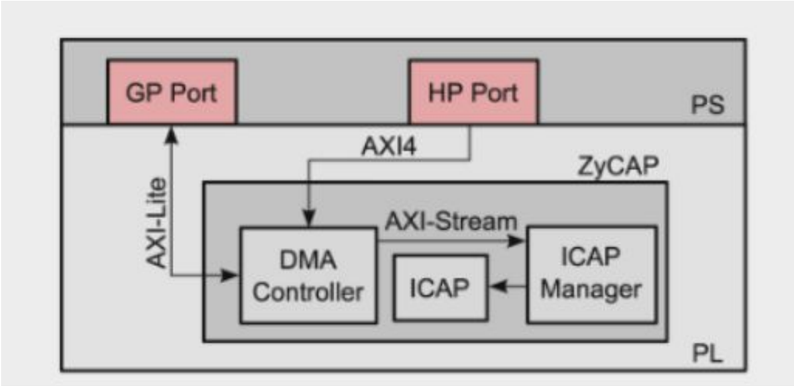


Fig. 9. ZyPR Linux Stack [8].

Performance: ZyCAP

- Authors: Kizheppatt Vipin, Suhaib A. Fahmy
  - Publication: IEEE Embedded Systems Letters, 2014
- A PR configuration controller based on Zynq SoCs
- High throughput via soft DMA and ICAP (which has a 400 MB/s throughput)
- Provides drivers that can be used with Linux for managing and monitoring the PR process
- The controller executes independently from the processor ie the processor is free to execute it's own tasks instead of stalling until the reconfiguration process completes
- Uses the DRAM as a cache for storing the partial bitstreams. Uses LRU policy



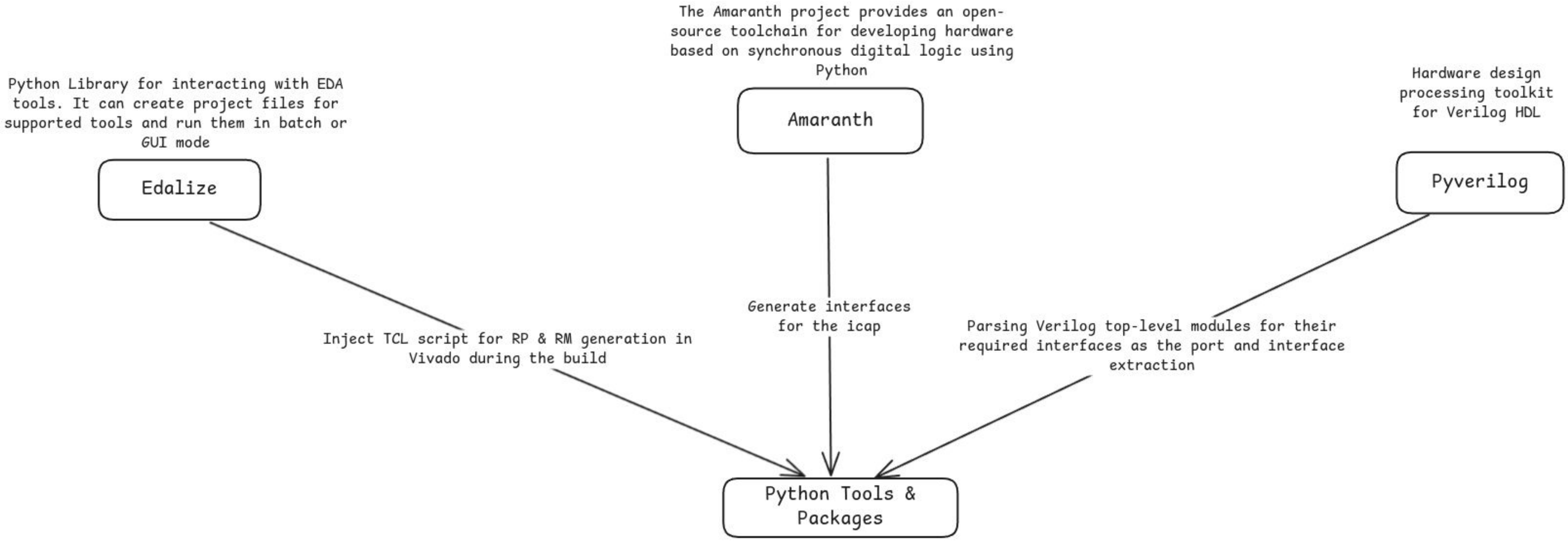
Architecture

COMPARISON OF RESOURCE UTILIZATION FOR DIFFERENT PR METHODS ON THE ZYNQ.

Method	Resource Utilisation			Throughput
	FFs	LUTs	BRAMs	(MBytes/sec)
PCAP	0	0	0	128
Xilinx ICAP (non-DMA)	443	296	0	19
Xilinx ICAP (with DMA)	443	296	0	67
ZyCAP	806	620	0	382

Utilisation & Performance

Abstraction: Python Packages



ZyPR Runtime APIs

- `ZyPR z(hardware,configs)` – ZyPR constructor takes overrides for default bitstream and configuration directories.
- `z.init(config)` – load a default configuration into the PL. This uses the FPGA manager driver, as the initialising bitstreams must be loaded over PCAP before the ICAP can be used.
- `z.status()` – returns a struct containing the status of the PL including the currently loaded `string:config`, `string:mode`, and `bool:pl_busy`.
- `z.configs()` – returns the available configurations in the default location.
- `z.config(config)` – load a configuration into the PL.
- `z.alloc(size, name)` – allocates contiguous memory for accelerator use.
- `z.exit()` – cleanly tears down the ZyPR runtime.
- `s.modes()` – returns the available modes within the configuration.
- `s.mode(mode)` – load a mode into a configuration.
- `s.write(reg)` – read from a register specified in the mode.
- `s.read(reg)` – write to a register specified in the mode.
- `s.transfer(buffer, type, direction)` – read/write a buffer in PS to a configuration in the PL either via DMA stream or a memory transfer. `enum:type` may either be `dma` or `axi`. `enum:direction` may also be a `two_way_transfer`, which sends data from the PS to the PL and waits for the PL to write back into the PS.

Comparison

Table 1. Build Tool Comparison

Framework	High Level Spec.	Partitioning	Floorplanning	Phys. Impl.	HLS Support	ZynqMP Support	Custom Static Logic	Device Tree Config	Driver Automation	Post Build Modules	Bitstream Prep.	Custom Root FS
Xilinx Vivado	○	○	◐	●	○	●	◐	◐	○	○	○	○
Intel Quartus	○	○	◐	●	○	○	◐	◐	○	○	○	○
ReconOS	○	○	○	○	○	○	◐	◐	○	○	●	◐
FOS	●	◐	○	○	◐	●	◐	◐	◐	●	●	●
ARTICo^3	●	◐	○	○	◐	●	◐	◐	◐	●	●	●
CoPR	●	●	◐	○	○	○	○	○	○	○	○	○
ZyPR	●	◐	◐	○	●	●	●	●	●	●	●	●

● : The step is fully automated by the tool requiring no designer intervention. ○ : No automation in this operation. ◐ : Partial automation is provided by tool.

Table 2. Runtime Comparison

Runtime	Linux Support	Bit. Management	Non-Blocking PR	Multi-User	ICAP Support	ZynqMP Support	PR Scheduler	Userspace API	Generic Drivers
FPGA Manager	●	○	○	○	○	●	◐	◐	○
ReconOS*	●	●	○	○	●	○	●	◐	○
FOS*	●	●	○	●	◐	●	●	●	◐
ARTICo^3*	●	●	○	●	○	●	●	●	◐
ZyCAP	○	●	●	○	●	○	◐	○	○
ZyPR	●	●	●	○	●	●	◐	●	●

● : Fully supported. ○ : Unsupported. ◐ : Partial support or allows other tools to implement. *Built on top of FPGA Manager.

- **ReconOS**:
  - Future PR modules would require the kernel to be recompiled with new bitstreams and associated drivers
- **ARTICo**:
  - Kernels must conform to a shell-defined interface, meaning accelerators need to be designed to fit the shell interfaces and their runtime API
- **CoPR**:
  - An previous framework (based on ISE) that automated partitioning and creating APIs for reconfiguration

Utilisation

Table 3. PR Manager Static PL Resources

PR Region Interfaces	FFs	LUTs	BRAMs	Total LUT Utilisation of PL (%)
1 AXI4-Lite (32-bit) + 0 AXI4-Stream (32-bit)	7,688	5,132	5	7.27
2 AXI4-Lite (32-bit) + 0 AXI4-Stream (32-bit)	7,738	5,142	5	7.28
1 AXI4-Lite (32-bit) + 1 AXI4-Stream (32-bit)	8,817	5,619	5	7.96
2 AXI4-Lite (32-bit) + 2 AXI4-Stream (32-bit)	9,063	6,051	5	8.58

Evaluated on Xilinx Zynq UltraScale+ MPSoC ZU3EG

Varying custom accelerators requiring various interface widths

Example (Case Study)

Example Program and spec.json:

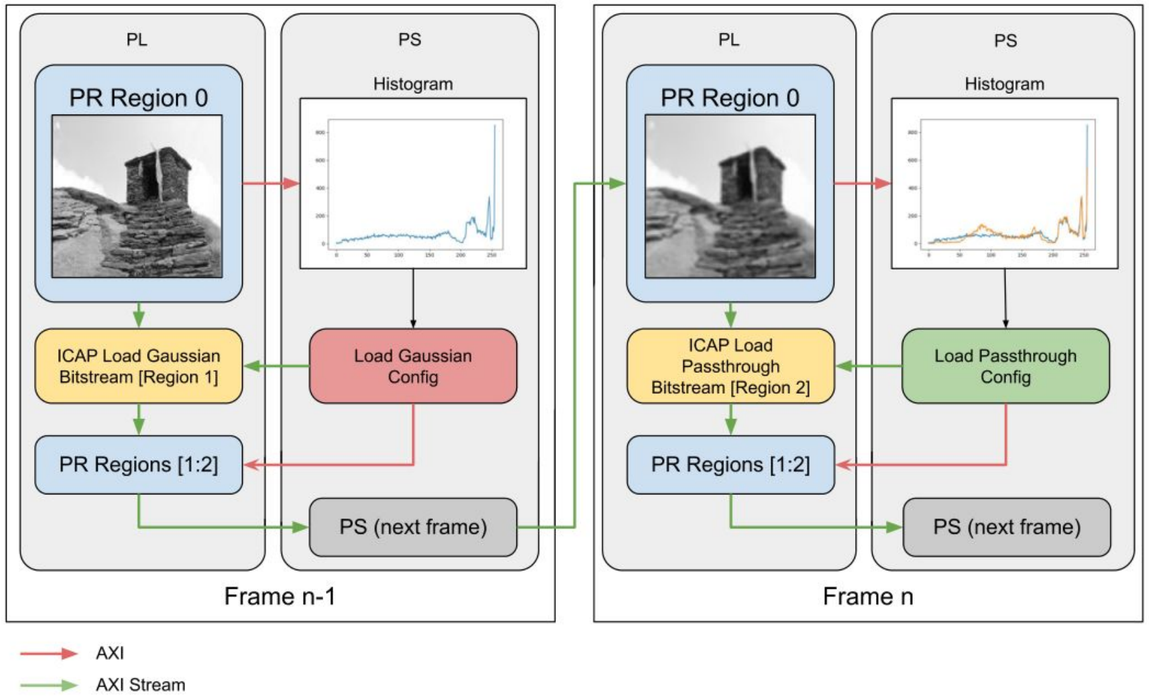
```cpp
#include <zypr.h>

void main()
{
    string hardware = "/lib/firmware/";
    string configs = "/lib/configs/";

    Zypr z(hardware, configs);

    /* `load` writes to the AXI-Stream
    Switch to set the multiplexed
    outputs as well as writes the
    default MMIO register values */
    cout << z.configs() << endl;
    Config s = z.config("gaussian");

    /* `mode` writes to MMIO registers
    with any custom settings specified
    by the designer in the generated
    JSON objects. */
    cout << s.modes() << endl;
    s.mode("full_hd");
}
```

Listing 5. C++ API.

```json
"slug": "gaussian",
"type": "partial",
"regions": [
    {
        "name": "gaussian_a",
        "bitstream": "gaussian_a.bin",
        "overlay": "gaussian.dtbo",
        "interfaces": {
            "axi_stream": "0x1",
            "axi_mmio": "0xa0050000"
        },
        "modes": {
            "default": {
                "0x40": "0x438",
                "0x44": "0x780"
            },
            "full_hd": {
                "0x40": "0x1E0",
                "0x44": "0x280"
            }
        }
    }
]
```

Listing 6. JSON Config and Mode.

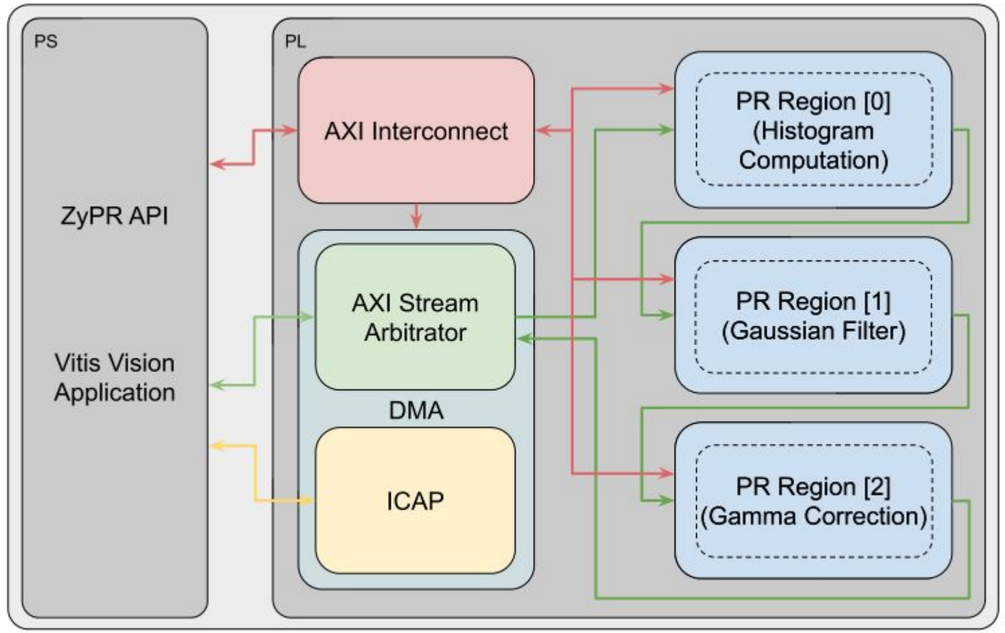
Flow:



Case study was done by deploying an HLS Vitis Vision accelerated image processing application using a USB webcam input (@ 30 fps or new frame at every 33.3 ms)

Example Output

Created Output:



Throughput Performance:

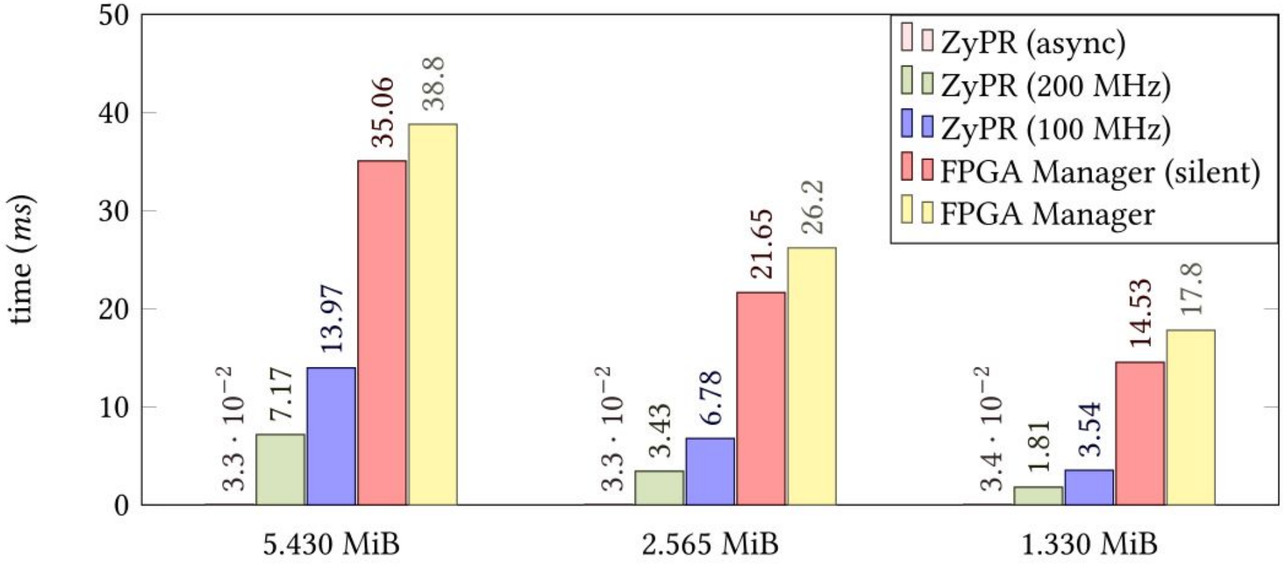


Fig. 12. PR Runtime Performance (time to load bitstream).

Reconfiguration time for ZyPR: 1.808 ms, FPGA Manager: 17.8 ms

Thank you